

OOP Interview Questions and Answers

1. What is Object-Oriented Programming (OOP)?

Interviewer: What is Object-Oriented Programming?

Candidate: OOP stands for Object-Oriented Programming. It is a programming approach where we organize code using classes and objects. In my understanding, OOP helps us make applications more modular, reusable, and easier to maintain. For example, in a banking application, we can create a Customer class and an Account class to represent real-world entities. This approach reduces code duplication and makes development more structured.

2. What is a Class?

Interviewer: Can you explain what a class is?

Candidate: A class is a blueprint or template used to create objects. It defines the properties and behaviors that objects will have. For example, if we have a Student class, it may contain attributes like name and roll number, and methods like study() and attendClass(). The class itself does not hold actual data; it only defines the structure.

3. What is an Object?

Interviewer: What is an object?

Candidate: An object is an instance of a class. It represents a real-world entity and contains actual values for the attributes defined in the class. For example, if Student is a class, then "Rahul" and "Priya" can be individual objects of that class. Objects allow us to access the data and methods defined within the class.

4. What are the main principles of OOP?

Interviewer: What are the four pillars of OOP?

Candidate: The four main pillars of OOP are Encapsulation, Inheritance, Polymorphism, and Abstraction. Encapsulation helps secure data, Inheritance promotes code reuse, Polymorphism allows one interface to have multiple forms, and Abstraction hides implementation details from users. Together, these concepts make software more scalable and maintainable.

5. What is Encapsulation?

Interviewer: Explain Encapsulation with an example.

Candidate: Encapsulation is the process of binding data and methods together within a single unit, usually a class, while restricting direct access to the data. For example, in a BankAccount class, the balance variable can be private, and users can access it only through methods like deposit() and withdraw(). This helps protect data from unauthorized modifications.

6. What is Inheritance?

Interviewer: What do you understand by Inheritance?

Candidate: Inheritance allows one class to acquire the properties and behaviors of another class. It promotes code reusability and reduces redundancy. For example, if we have a Vehicle class with common properties like speed and fuelType, then Car and Bike classes can inherit these properties. This way, we don't need to rewrite the same code multiple times.

7. What is Polymorphism?

Interviewer: Can you explain Polymorphism?

Candidate: Polymorphism means "many forms." It allows the same method or interface to behave differently depending on the situation. For example, a method called calculateSalary() may work differently for a

FullTimeEmployee and a PartTimeEmployee. Polymorphism makes code flexible and easier to extend.

8. What is Abstraction?

Interviewer: What is Abstraction?

Candidate: Abstraction is the process of hiding implementation details and showing only the essential functionality to the user. A real-world example is driving a car. We use the steering wheel, brake, and accelerator without knowing the internal engine mechanism. Similarly, in programming, abstraction helps reduce complexity and improves security.

9. What is a Constructor?

Interviewer: What is a constructor and why is it used?

Candidate: A constructor is a special method that gets called automatically when an object is created. Its main purpose is to initialize object data. For example, when creating a Student object, a constructor can automatically assign values like name and roll number. This ensures the object starts with valid data.

10. What is Method Overloading?

Interviewer: What is Method Overloading?

Candidate: Method Overloading is a type of compile-time polymorphism where multiple methods have the same name but different parameters. For example, an add() method can accept two integers or three integers. The compiler decides which method to execute based on the arguments provided. It improves readability and flexibility.

11. What is Method Overriding?

Interviewer: What is Method Overriding?

Candidate: Method Overriding occurs when a child class provides its own implementation of a method already defined in the parent class. For example, a Vehicle class may have a start() method, and a Car class can override it with its own specific behavior. This is a key feature of runtime polymorphism.

12. What is the difference between Method Overloading and Method Overriding?

Interviewer: Can you differentiate between Method Overloading and Method Overriding?

Candidate: Yes. Method Overloading happens within the same class and requires different parameter lists. It is resolved during compile time. Method Overriding occurs between parent and child classes where the method signature remains the same, and it is resolved during runtime. So, Overloading provides flexibility, while Overriding enables customized behavior.

13. What are Access Modifiers?

Interviewer: What are Access Modifiers?

Candidate: Access Modifiers control the visibility and accessibility of class members. Common access modifiers are Public, Private, Protected, and Default. For example, private variables can only be accessed within the same class, which helps achieve encapsulation and data security.

14. What is an Interface?

Interviewer: What is an Interface?

Candidate: An Interface is a blueprint that contains method declarations without implementation. Any class implementing the interface must provide

implementations for those methods. For example, a Payment interface may contain a makePayment() method, and classes like CreditCardPayment and UPIpayment can implement it differently. Interfaces help achieve abstraction and multiple inheritance.

15. What is an Abstract Class?

Interviewer: What is an Abstract Class?

Candidate: An Abstract Class is a class that cannot be instantiated directly. It can contain both abstract methods and normal methods. For example, an Employee abstract class may define common functionality while leaving salary calculation to child classes. It is useful when some behavior is common and some behavior differs among subclasses.

16. What is the difference between an Interface and an Abstract Class?

Interviewer: How would you compare an Interface and an Abstract Class?

Candidate: An Interface is mainly used to define a contract that multiple classes can follow, while an Abstract Class can provide both implementation and abstraction. If classes share common functionality, I would use an Abstract Class. If unrelated classes need to follow the same behavior, I would use an Interface.

17. What is Association, Aggregation, and Composition?

Interviewer: Can you explain Association, Aggregation, and Composition?

Candidate: Yes.

- Association represents a relationship between two independent objects.
- Aggregation is a weak "has-a" relationship where objects can exist independently.

- Composition is a strong "has-a" relationship where the child object's lifecycle depends on the parent.

For example, a Department and Teacher relationship can be Aggregation because teachers can exist without a department. A House and Room relationship is Composition because rooms are part of the house and generally don't exist independently.

18. What is the Static Keyword?

Interviewer: What is the purpose of the Static Keyword?

Candidate: The static keyword is used for members that belong to the class rather than individual objects. For example, if all employees work in the same company, the company name can be declared as static because it is shared among all employee objects. Static members help save memory and provide common data across objects.

19. What is the difference between This and Super Keywords?

Interviewer: Explain the use of this and super keywords.

Candidate: The this keyword refers to the current object of the class. It is commonly used to differentiate instance variables from local variables. The super keyword refers to the parent class object and is used to access parent class methods, variables, or constructors. In inheritance scenarios, super helps reuse parent class functionality efficiently.

20. What are the advantages of OOP?

Interviewer: Why do companies prefer OOP?

Candidate: Companies prefer OOP because it makes software development more organized, scalable, and maintainable. OOP promotes code reuse through inheritance, improves security through encapsulation, reduces complexity through abstraction, and provides flexibility through

polymorphism. In large projects with multiple developers, OOP helps manage code efficiently and makes future enhancements easier.